

floating point literal contd...

```
Test8.java
-----
//Range and size of floating point literal
void main()
{
    IO.println("\n Float range:");
    IO.println(" min: " + Float.MIN_VALUE);
    IO.println(" max: " + Float.MAX_VALUE);
    IO.println(" size :"+Float.SIZE);

    IO.println("\n Double range:");
    IO.println(" min: " + Double.MIN_VALUE);
    IO.println(" max: " + Double.MAX_VALUE);
    IO.println(" size :"+Double.SIZE);
}
```

```
Test9.java
-----
void main()
{
    double d1 = 10.0/3.0;

    IO.println(d1);

    //How to print 2 digit after decimal
    System.out.printf("Value is %.2f",d1); //don't use + operator

    IO.println();
}
```

**What is signed and unsigned integer ?**

- \* A signed integer can hold **positive & negative** both kind of values.  
Example of signed integer : byte, short, int, long, float and double data type.
- \* An unsigned integer can hold only **positive** value including 0.  
Example of unsigned integer : char data type

**Character Literal :**

- \* It is also known as char literal which represents unsigned integer.
- \* In character literal, We have only one data type i.e **char data type which accepts 16 bits of memory.**
- \* We can represent character literal by following ways :

a) Single character OR Single Digit OR A Single Special character enclosed with single quotes.

```
Example :
char c1 = 'A';
char c2 = '5';
char c3 = '@';
```

```
char c1 = 'AB'; //Invalid
char c2 = ""; //Invalid [Empty Character Literal]
```

\* In older languages like C and C++, which supports ASCII format and range is 0 - 255 (total 256 chacaters) but java supports UNICODE format (ASCII + NON ASCII) and range is **0 to 65535**.

```
Example :
char ch = 65535; //Valid
char x = 65536; //Invalid out of the range of UNICODE
```

```
byte b = (byte) 12;    //-128 to 127
short s = (short) 125; //-32768 to 32767
char c = (char) 100;   //0 to 65535
```

\* A character literal we can assign to integral literal to know the UNICODE value of that particular character.

```
Example :
int x = 'A';
```

\* All escape sequences we can represent as character literal.

```
Example :
char ch = '\n';
```

\* A chacater literal we can also represent in 4 digit UNICODE hexadecimal number. The range is as follows :

Format is : '\uXXXX'  
Here u (small u) represents UNICODE  
X represents exactly 4 digits

Minumum Range : '\u0000'  
Maximum Range : '\uffff'

Example :  
The UNICODE /ASCII value of character 'A' is 65.

65 is a decimal number. If I want to represent in 4 digit hexadecimal format, where the format is

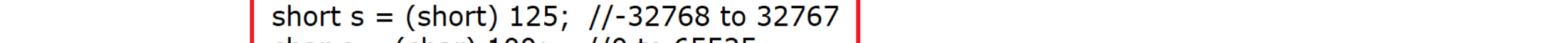
```
'\u0000'
'\uffff'
```

(6 5)<sub>10</sub> = ( ? )<sub>16</sub>

|    |    |   |
|----|----|---|
| 16 | 65 | 1 |
| 16 | 4  | 4 |
|    | 0  |   |

(65)<sub>10</sub> = (041)<sub>16</sub>

But 041 is a 3 digit number so It will generate an error because the format is , **4 digit number**



```
byte b = (byte) 12;    //-128 to 127
short s = (short) 125; //-32768 to 32767
char c = (char) 100;   //0 to 65535
```

**Note : byte, short and char are below than int so to assign an int value, implicitly java compiler performs explicit type casting OR Narrowing but the int value must be within the range only.**

```
//Programs :
-----
void main()
{
    char ch1 = 'a';
    IO.println("ch1 value is :"+ch1);

    char ch2 = '5';
    IO.println("ch2 value is :"+ch2);

    char ch3 = '$';
    IO.println("ch3 value is :"+ch3);
}
```

```
Test2.java
-----
void main()
{
    int ch = 'A';
    IO.println("ch value is :"+ch);

    char ch1 = 'A';
    IO.println("ch1 value is :"+ch1);

    char ch2 = 97;
    IO.println("ch2 value is :"+ch2);
}
```

```
Test3.java
-----
//The UNICODE value for ? character is 63
void main()
{
    char ch1 = 63;
    IO.println("ch1 value is :"+ch1);

    char ch2 = 64;
    IO.println("ch2 value is :"+ch2);

    char ch3 = 65;
    IO.println("ch3 value is :"+ch3);

    char ch4 = 01;
    IO.println("ch4 value is :"+ch4);
}
```